Kernel v3.3

Security Audit

audited by Prune Report

Audit Info

version : 2.0
delivered at : 2025 Mar 2
delivered by : Felix Kim

About Prune Report

Prune Report is a network of smart contract security experts and CTF players, united by a commitment to delivering top-tier auditing services on-demand and with unwavering accuracy. Our expertise extends across the blockchain landscape, encompassing smart contracts in Solidity, Vyper, and Rust, as well as comprehensive frontend, backend, and node audits.

Severity Criteria

This report categorizes vulnerabilities based on their potential impact and likelihood of exploitation:

Severity	Criteria
Critical	Issues that threaten the system's integrity, allowing asset loss, full disruption, or unauthorized control. Immediate resolution is required.
High	Vulnerabilities that significantly impact functionality or trust, potentially causing financial loss or system instability.
Medium	Issues causing localized disruptions or incorrect outputs, affecting specific users or functions without widespread damage.
Low	Minor inefficiencies or optimization opportunities with minimal impact on operations or security.
Informational	Observations or recommendations to enhance code quality, maintainability, or adherence to best practices.

Summary

Target Code

Github Link : https://github.com/zerodevapp/kernel
Audit Scope
- Kernel v3.3 Incremental Audit
- https://github.com/zerodevapp/kernel/pull/129

Code Quality

Contract Documentation Quality: Partial
The contract includes some level of documentation, but certain areas lack inline comments or detailed explanations. Improving documentation—particularly in key functions and critical logic paths—would enhance readability and maintainability. Proper comments and documentation help future developers or maintainers quickly understand the code, reducing the risk of misinterpretation or errors during updates or reviews.

Test Quality: Superior
The tests are also very well organized and show excellent quality in terms of accuracy and stability of the code. They verify various scenarios and take into account both general and exceptional situations.

Reviewer Note
Kernel v3.3 supports the EIP-7702 feature, which introduces a new transaction type allowing EOAs to temporarily adopt smart contract behavior. In addition to supporting EIP-7702, Kernel v3.3 includes refactoring efforts to improve maintainability and efficiency. The test suite is exceptionally well-structured, demonstrating high accuracy and stability. It effectively verifies various scenarios, covering both standard and edge cases to ensure robust contract behavior.

Issue	Severity	Status
Potential Compatibility Issue in Kernel Contract's EIP-7702 Detection Logic	**Informational**	**Fix**
Inefficient Gas Usage in changeRootValidator for EIP-7702 EOAs	**Informational**	**Ack**

Issues

1. Informational - Potential Compatibility Issue in Kernel Contract's EIP-7702 Detection Logic

Solidity

```
    function initialize(
        ValidationId _rootValidator,
        IHook hook,
        bytes calldata validatorData,
        bytes calldata hookData,
        bytes[] calldata initConfig
    ) external {
        ValidationStorage storage vs = _validationStorage();
        if(
            ValidationId.unwrap(vs.rootValidator) != bytes21(0) || bytes2(address(this).code)
== EIP7702_PREFIX) {
            revert AlreadyInitialized();
        }
        if (ValidationId.unwrap(_rootValidator) == bytes21(0)) {
            revert InvalidValidator();
        }
        ValidationType vType = ValidatorLib.getType(_rootValidator);
        if (vType != VALIDATION_TYPE_VALIDATOR && vType != VALIDATION_TYPE_PERMISSION) {
            revert InvalidValidationType();
        }
        _setRootValidator(_rootValidator);
        ValidationConfig memory config = ValidationConfig({nonce: uint32(1), hook: hook});
        vs.currentNonce = 1;
        _installValidation(_rootValidator, config, validatorData, hookData);
        for (uint256 i = 0; i < initConfig.length; i++) {
            (bool success,) = address(this).call(initConfig[i]);
            if (!success) {
                revert InitConfigError(i);
            }
        }
    }
```

The Kernel contract's initialize function verifies whether the current address is an EOA with EIP-7702 applied by checking the first 2 bytes of the contract's code:

However, EIP-7702 defines its delegation designator as 3 bytes (**0xef0100** || address), meaning the check only verifies **0xef01**, missing the last byte.

Currently, this does not cause functional issues since 0xef is not a permitted opcode, and 0xef01 prefix is only used by EIP-7702. However, if another standard in the future introduces a different 0xef01 prefixed designator, this check may lead to unintended compatibility issues.

[Recommendation]
Modify the verification logic to check all 3 bytes of the EIP-7702 prefix (**0xef0100**), ensuring future compatibility.

[Update]
v2.0 - Fixed
This issue has been resolved in commit 53290c001b4f7dbf02697b84409e543bb2751a35, which correctly defines the EIP-7702 prefix.

2. Informational - Inefficient Gas Usage in changeRootValidator for EIP-7702 EOAs

Solidity

```
    function changeRootValidator(
        ValidationId _rootValidator,
        IHook hook,
        bytes calldata validatorData,
        bytes calldata hookData
    ) external payable onlyEntryPointOrSelfOrRoot {
        ValidationStorage storage vs = _validationStorage();
        if (ValidationId.unwrap(_rootValidator) == bytes21(0)) {
            revert InvalidValidator();
        }
        ValidationType vType = ValidatorLib.getType(_rootValidator);
        if (vType != VALIDATION_TYPE_VALIDATOR && vType != VALIDATION_TYPE_PERMISSION) {
            revert InvalidValidationType();
        }
        _setRootValidator(_rootValidator);
        if (_validationStorage().validationConfig[_rootValidator].hook ==
IHook(HOOK_MODULE_NOT_INSTALLED)) {
            // when new rootValidator is not installed yet
            ValidationConfig memory config = ValidationConfig({nonce: uint32(vs.currentNonce),
hook: hook});
            _installValidation(_rootValidator, config, validatorData, hookData);
        }
    }
```

With EIP-7702, an EOA is temporarily converted into a smart contract by registering Kernel code directly on the EOA, rather than interacting with the Kernel contract externally.

Since the `Kernel.initialize()` function is never called for EIP-7702 EOAs, the currentNonce value remains 0, which causes `ValidationManager._installValidation()` to fail due to unmet nonce conditions when `Kernel.changeRootValidator()` is called.

If changing the root validator is not intended for EIP-7702 EOAs, it is recommended to add an early check for the EIP-7702 prefix at the beginning of the function. This would prevent unnecessary gas consumption by reverting before executing any further logic.

[Recommendation]
Introduce an early check for the EIP-7702 prefix at the start of `Kernel.changeRootValidator()` to efficiently revert if executed under an EIP-7702 context.

[Update]
v2.0 - Acknowledgement
The ZeroDevApp team has acknowledged this issue, but it remains unresolved due to code size limitations.